

Week 2 at a glance

We will be learning and practicing to:

- Model systems with tools from discrete mathematics and reason about implications of modelling choices. Explore applications in CS through multiple perspectives, including software, hardware, and theory.
 - Selecting and representing appropriate data types and using notation conventions to clearly communicate choices
 - Determining the properties of positional number representations, including overflow and bit operations
- Translate between different representations to illustrate a concept.
 - Translating between symbolic and English versions of statements using precise mathematical language
 - Tracing algorithms specified in pseudocode
 - Representing numbers using positional representations, including decimal, binary, hexadecimal, fixed-width representations, and 2s complement
- Use precise notation to encode meaning and present arguments concisely and clearly
 - Precisely describing a set using appropriate notation e.g. roster method, set builder notation, and recursive definitions
 - Defining functions using multiple representations
- Know, select and apply appropriate computing knowledge and problem-solving techniques. Reason about computation and systems. Use mathematical techniques to solve problems. Determine appropriate conceptual tools to apply to new situations. Know when tools do not apply and try different approaches. Critically analyze and evaluate candidate solutions.
 - Using a recursive definition to evaluate a function or determine membership in a set
 - Using the definitions of the div and mod operators on integers

TODO:

#FinAid Assignment on Canvas (complete as soon as possible)

Review quiz based on Week 1 class material (due Monday 01/12/2026)

Homework assignment 1 (due Thursday 01/15/2026)

Review quiz based on Week 2 class material (due Monday 01/19/2026)

Week 2 Monday: Sets, functions, and algorithms

Let's practice with functions related to some of our applications so far.

Recall: We model the collection of user ratings of the four movies Superman, Wicked, KPop Demon Hunters, A Minecraft Movie as the set $\{-1, 0, 1\}^4$. One function that compares pairs of ratings is

$$d_0 : \{-1, 0, 1\}^4 \times \{-1, 0, 1\}^4 \rightarrow \mathbb{R}$$

given by

$$d_0((x_1, x_2, x_3, x_4), (y_1, y_2, y_3, y_4)) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2 + (x_4 - y_4)^2}$$

Notice: any ordered pair of ratings is an okay input to d_0 .

Notice: there are (at most)

$$(3 \cdot 3 \cdot 3 \cdot 3) \cdot (3 \cdot 3 \cdot 3 \cdot 3) = 3^8 = 6561$$

many pairs of ratings. There are therefore lots and lots of real numbers that are not the output of d_0 .

Recall: RNA is made up of strands of four different bases that encode genomic information in specific ways. The bases are elements of the set $B = \{\mathbf{A}, \mathbf{C}, \mathbf{U}, \mathbf{G}\}$. The set of RNA strands S is defined (recursively) by:

$$\begin{array}{ll} \text{Basis Step:} & \mathbf{A} \in S, \mathbf{C} \in S, \mathbf{U} \in S, \mathbf{G} \in S \\ \text{Recursive Step:} & \text{If } s \in S \text{ and } b \in B, \text{ then } sb \in S \end{array}$$

where sb is string concatenation.

Pro-tip: informal definitions sometime use $\dots$ to indicate “continue the pattern”. Often, to make this pattern precise we use recursive definitions.

Name	Domain	Codomain	Rule	Example
<i>rnalen</i>	S	$\mathbb{Z}^+$	Basis Step: If $b \in B$ then $rnalen(b) = 1$ Recursive Step: If $s \in S$ and $b \in B$, then $rnalen(sb) = 1 + rnalen(s)$	$rnalen(\mathbf{AC}) \overset{\text{rec step}}{=} 1 + rnalen(\mathbf{A})$ $\overset{\text{basis step}}{=} 1 + 1 = 2$
<i>basecount</i>	$S \times B$	$\mathbb{N}$	Basis Step: If $b_1 \in B, b_2 \in B$ then $basecount((b_1, b_2)) =$ $\begin{cases} 1 & \text{when } b_1 = b_2 \\ 0 & \text{when } b_1 \neq b_2 \end{cases}$ Recursive Step: If $s \in S, b_1 \in B, b_2 \in B$ $basecount((sb_1, b_2)) =$ $\begin{cases} 1 + basecount((s, b_2)) & \text{when } b_1 = b_2 \\ basecount((s, b_2)) & \text{when } b_1 \neq b_2 \end{cases}$	$basecount((\mathbf{ACU}, \mathbf{C})) =$
“2 to the power of”	$\mathbb{N}$	$\mathbb{N}$	Basis Step: $2^0 = 1$ Recursive Step: If $n \in \mathbb{N}, 2^{n+1} =$	
“ b to the power of i ”	$\mathbb{Z}^+ \times \mathbb{N}$	$\mathbb{N}$	Basis Step: $b^0 = 1$ Recursive Step: If $i \in \mathbb{N}, b^{i+1} = b \cdot b^i$	

$2^0 = 1$	$2^1 = 2$	$2^2 = 4$	$2^3 = 8$	$2^4 = 16$	$2^5 = 32$	$2^6 = 64$	$2^7 = 128$	$2^8 = 256$	$2^9 = 512$	$2^{10} = 1024$
-----------	-----------	-----------	-----------	------------	------------	------------	-------------	-------------	-------------	-----------------

Integer division and remainders (aka The Division Algorithm) Let n be an integer and d a positive integer. There are unique integers q and r , with $0 \leq r < d$, such that $n = dq + r$. In this case, d is called the divisor, n is called the dividend, q is called the quotient, and r is called the remainder.

Because these numbers are guaranteed to exist, the following functions are well-defined:

div : $\mathbb{Z} \times \mathbb{Z}^+ \rightarrow \mathbb{Z}$ given by **div** ((n, d)) is the quotient when n is the dividend and d is the divisor.

mod : $\mathbb{Z} \times \mathbb{Z}^+ \rightarrow \mathbb{Z}$ given by **mod** ((n, d)) is the remainder when n is the dividend and d is the divisor.

Because these functions are so important, we sometimes use the notation $n \text{ div } d = \text{div} ((n, d))$ and $n \text{ mod } d = \text{mod} ((n, d))$.

Pro-tip: The functions **div** and **mod** are similar to (but not exactly the same as) the operators $/$ and $\%$ in Java and python.

Example calculations:

$20 \text{ div } 4$

$20 \text{ mod } 4$

$20 \text{ div } 3$

$20 \text{ mod } 3$

$-20 \text{ div } 3$

$-20 \text{ mod } 3$

Week 2 Wednesday: Representing numbers

Modeling uses data-types that are encoded in a computer. The details of the encoding impact the efficiency of algorithms we use to understand the systems we are modeling and the impacts of these algorithms on the people using the systems. Case study: how to encode numbers?

Definition For b an integer greater than 1 and n a positive integer, the **base b expansion of n** is

$$(a_{k-1} \cdots a_1 a_0)_b$$

where k is a positive integer, $a_0, a_1, \dots, a_{k-1}$ are (symbols for) nonnegative integers less than b , $a_{k-1} \neq 0$, and

$$n = \sum_{i=0}^{k-1} a_i b^i$$

Notice: *The base b expansion of a positive integer n is a string over the alphabet $\{x \in \mathbb{N} \mid x < b\}$ whose leftmost character is nonzero.*

Base b	Collection of possible coefficients in base b expansion of a positive integer
Binary ($b = 2$)	$\{0, 1\}$
Ternary ($b = 3$)	$\{0, 1, 2\}$
Octal ($b = 8$)	$\{0, 1, 2, 3, 4, 5, 6, 7\}$
Decimal ($b = 10$)	$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
Hexadecimal ($b = 16$)	$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$ letter coefficient symbols represent numerical values $(A)_{16} = (10)_{10}$ $(B)_{16} = (11)_{10}$ $(C)_{16} = (12)_{10}$ $(D)_{16} = (13)_{10}$ $(E)_{16} = (14)_{10}$ $(F)_{16} = (15)_{10}$

Examples:

$(1401)_2$

$(1401)_{10}$

$(1401)_{16}$

New! An algorithm is a finite sequence of precise instructions for solving a problem.

Algorithms can be expressed in English or in more formalized descriptions like pseudocode or fully executable programs.

Sometimes, we can define algorithms whose output matches the rule for a function we already care about. Consider the (integer) logarithm function

$$\log b : \{b \in \mathbb{Z} \mid b > 1\} \times \mathbb{Z}^+ \rightarrow \mathbb{N}$$

defined by

$\log b((b, n)) =$ greatest integer y so that b^y is less than or equal to n

Calculating integer part of base b logarithm

```

1  procedure  $\log b(b, n)$ : positive integers with  $b > 1$ )
2     $i := 0$ 
3    while  $n > b - 1$ 
4       $i := i + 1$ 
5       $n := n \text{ div } b$ 
6    return  $i$  { $i$  holds the integer part of the base  $b$  logarithm of  $n$ }

```

Trace this algorithm with inputs $b = 3$ and $n = 17$

	b	n	i	$n > b - 1?$
Initial value	3	17		
After 1 iteration				
After 2 iterations				
After 3 iterations				

Compare: does the output match the rule for the (integer) logarithm function?

Two algorithms for constructing base b expansion from decimal representation

Most significant first: Start with left-most coefficient of expansion (highest value)

Informally: Build up to the value we need to represent in “greedy” approach, using units determined by base.

Calculating base b expansion, from left

```
1 procedure basebmost( $n, b$ : positive integers with  $b > 1$ )
2    $v := n$ 
3    $k := 1 + \text{output of } \log b \text{ algorithm with inputs } b \text{ and } n$ 
4   for  $i := 1$  to  $k$ 
5      $a_{k-i} := 0$ 
6     while  $v \geq b^{k-i}$ 
7        $a_{k-i} := a_{k-i} + 1$ 
8        $v := v - b^{k-i}$ 
9   return  $(a_{k-1}, \dots, a_0) \{ (a_{k-1} \dots a_0)_b \text{ is the base } b \text{ expansion of } n \}$ 
```

Least significant first: Start with right-most coefficient of expansion (lowest value)

Idea: (when $k > 1$)

$$\begin{aligned} n &= a_{k-1}b^{k-1} + \cdots + a_1b + a_0 \\ &= b(a_{k-1}b^{k-2} + \cdots + a_1) + a_0 \end{aligned}$$

so $a_0 = n \bmod b$ and $a_{k-1}b^{k-2} + \cdots + a_1 = n \operatorname{div} b$.

Calculating base b expansion, from right

```
1 procedure basebleast( $n, b$ : positive integers with  $b > 1$ )
2    $k := 0$ 
3    $q := n$ 
4   while  $q \neq 0$ 
5      $a_k := q \bmod b$ 
6      $k := k + 1$ 
7      $q := q \operatorname{div} b$ 
8   return  $(a_{k-1}, \dots, a_0)\{(a_{k-1} \dots a_0)_b \text{ is the base } b \text{ expansion of } n\}$ 
```

Week 2 Friday: Algorithms for numbers

Find and fix any and all mistakes with the following:

- (a) $(1)_2 = (1)_8$
- (b) $(142)_{10} = (142)_{16}$
- (c) $(20)_{10} = (10100)_2$
- (d) $(35)_8 = (1D)_{16}$

Practice: write an algorithm for converting from base b_1 expansion to base b_2 expansion:

Definition For b an integer greater than 1, w a positive integer, and n a nonnegative integer _____, the **base b fixed-width w expansion of n** is

$$(a_{w-1} \cdots a_1 a_0)_{b,w}$$

where $a_0, a_1, \dots, a_{w-1}$ are nonnegative integers less than b and

$$n = \sum_{i=0}^{w-1} a_i b^i$$

Decimal $b = 10$	Binary $b = 2$	Binary fixed-width 10 $b = 2, w = 10$	Binary fixed-width 7 $b = 2, w = 7$	Binary fixed-width 4 $b = 2, w = 4$
$(20)_{10}$				

Definition For b an integer greater than 1, w a positive integer, w' a positive integer, and x a real number the **base b fixed-width expansion of x with integer part width w and fractional part width w'** is $(a_{w-1} \cdots a_1 a_0 . c_1 \cdots c_{w'})_{b,w,w'}$ where $a_0, a_1, \dots, a_{w-1}, c_1, \dots, c_{w'}$ are nonnegative integers less than b and

$$x \geq \sum_{i=0}^{w-1} a_i b^i + \sum_{j=1}^{w'} c_j b^{-j} \qquad \text{and} \qquad x < \sum_{i=0}^{w-1} a_i b^i + \sum_{j=1}^{w'} c_j b^{-j} + b^{-w'}$$

3.75 in fixed-width binary, integer part width 2, fractional part width 8	
0.1 in fixed-width binary, integer part width 2, fractional part width 8	

```

|welcome $jshell
| Welcome to JShell -- Version 10.0.1
| For an introduction type: /help intro

[jshell> 0.1
$1 ==>

[jshell> 0.2
$2 ==>

[jshell> 0.1 + 0.2
$3 ==>

[jshell> Math.sqrt(2)
$4 ==>

[jshell> Math.sqrt(2)*Math.sqrt(2)
$5 ==>

[jshell> █

```

Note: Java uses floating point, not fixed width representation, but similar rounding errors appear in both.

Representing negative integers in binary: Fix a positive integer width for the representation w , $w > 1$.

	To represent a positive integer n	To represent a negative integer $-n$
Sign-magnitude	$[0a_{w-2} \cdots a_0]_{s,w}$, where $n = (a_{w-2} \cdots a_0)_{2,w-1}$ Example $n = 17$, $w = 7$:	$[1a_{w-2} \cdots a_0]_{s,w}$, where $n = (a_{w-2} \cdots a_0)_{2,w-1}$ Example $-n = -17$, $w = 7$:
2s complement	$[0a_{w-2} \cdots a_0]_{2c,w}$, where $n = (a_{w-2} \cdots a_0)_{2,w-1}$ Example $n = 17$, $w = 7$:	$[1a_{w-2} \cdots a_0]_{2c,w}$, where $2^{w-1} - n = (a_{w-2} \cdots a_0)_{2,w-1}$ Example $-n = -17$, $w = 7$:

For positive integer n , to represent $-n$ in 2s complement with width w ,

- Calculate $2^{w-1} - n$, convert result to binary fixed-width $w - 1$, pad with leading 1, or
- Express $-n$ as a sum of powers of 2, where the leftmost 2^{w-1} is negative weight, or
- Convert n to binary fixed-width w , flip bits, add 1 (ignore overflow)

Challenge: use definitions to explain why each of these approaches works.

Representing 0:

So far, we have representations for positive and negative integers. What about 0?

	To represent a non-negative integer n	To represent a non-positive integer $-n$
Sign-magnitude	$[0a_{w-2} \cdots a_0]_{s,w}$, where $n = (a_{w-2} \cdots a_0)_{2,w-1}$ Example $n = 0$, $w = 7$:	$[1a_{w-2} \cdots a_0]_{s,w}$, where $n = (a_{w-2} \cdots a_0)_{2,w-1}$ Example $-n = 0$, $w = 7$:
2s complement	$[0a_{w-2} \cdots a_0]_{2c,w}$, where $n = (a_{w-2} \cdots a_0)_{2,w-1}$ Example $n = 0$, $w = 7$:	$[1a_{w-2} \cdots a_0]_{2c,w}$, where $2^{w-1} - n = (a_{w-2} \cdots a_0)_{2,w-1}$ Example $-n = 0$, $w = 7$: